



Fundamentals of Node.js

01

Now that we've built our shelter, it is time we spread the word about it. We are constantly seeking volunteers and donors. An attractive website would be an excellent way to garner some attention! I wish I could do this by myself!

You certainly can! It is extremely simple. Let me tell you how!



'JavaScript' is a scripting language that is used to create programs that operate within a web browser. Programs designed using JavaScript are also referred to as **'scripts'**.

The **'JavaScript'** programming language is a client-side scripting language. This implies that any script you write will be executed in the browser(client). JavaScript is mainly used to create dynamic websites. It allows interaction with user activities, such as clicking buttons, scrolling with the mouse, enlarging a thumbnail image, etc.

JS

Other uses of JavaScript are:

- To add, remove, and update HTML components dynamically.
- To play multimedia files.
- To create visual effects and animations.
- To perform form validations, etc.

What is Node.js?

‘**Node.js**’ is an open-source runtime environment that enables JavaScript code to be executed on the server. This way, the JavaScript code used to create the ‘Frontend’ can be also used for the ‘Backend (Server-Side)’ development.

The ‘Frontend’ is also known as the ‘**Client-Side**’. It refers to the part of the website that a user directly interacts with. Everything that a user views on a website, including the text, images, tables, graphs, animations, and navigation, belongs to the Frontend.

The ‘Backend’ is also known as the ‘**Server-Side**’. It refers to the part of the website that is not directly visible to the user. The backend is responsible for processing the data and activities of a user. It manages user requests and fetches data from the servers to be displayed at the frontend.

A few points to note regarding Node.js:

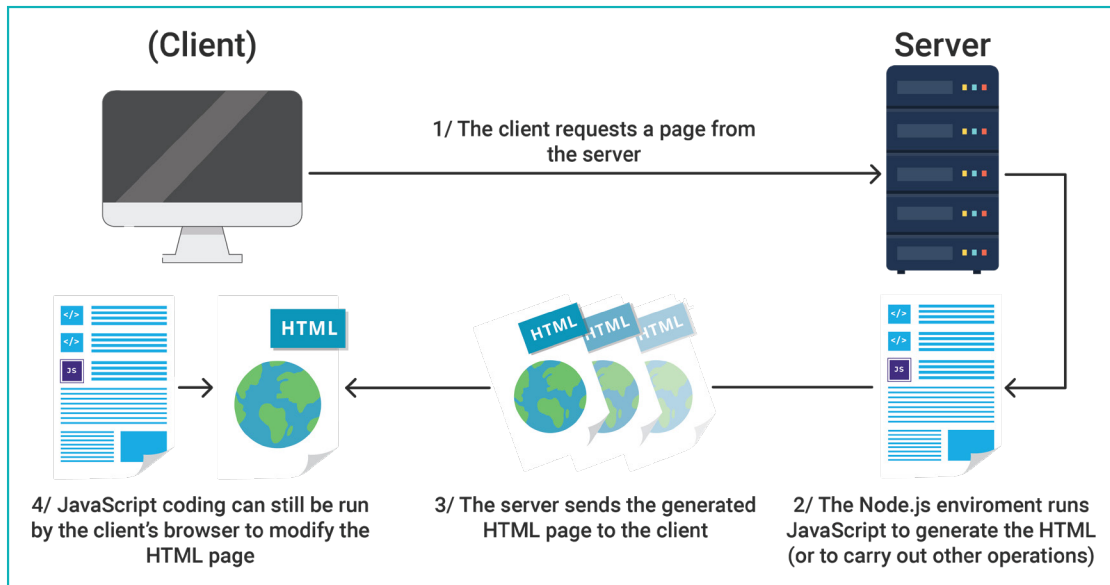
- It is available for free since it is Open Source.
- It can run on various platforms such as Windows, Linux, Mac OS, etc.
- It can generate dynamic content on a static webpage.

Note:

Node.js is NOT a programming language.

1. Fundamentals of Node.js

Look at the given figure to understand the use of Node.js with reference to the Client-Server Architecture.



Development of Node.js

The functionality of JavaScript is only available in browsers(client-side). Therefore, when developing a server-side application, we need to use a different server-side programming language, such as PHP, Java, Python, or .NET. As a result, web developers must know multiple languages when creating a full-fledged web application.

There is widespread use of different languages for building client-side and server-side software.

However, with the latest developments in web technologies, developers have also experimented with JavaScript to make it usable at the backend. The fact that developers can use the same language for both frontend and backend development simplifies the entire process.

This is one of the primary reasons for the development of Node.js.

Uses of Node.js

Node.js can be used to create different types of applications:

- **Streaming Web Applications:** Apps like Amazon Prime, Disney+Hotstar, and others allow you to stream movies, shows, and videos on a server.



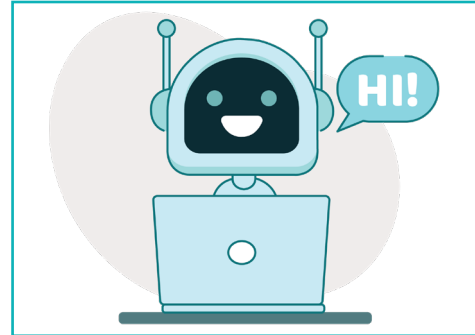
Image credits: Amazon

- **Social Media Websites:** Apps such as Facebook, Instagram, etc. allow you to connect with friends, family, and business associates anywhere in the world with the click of a button.

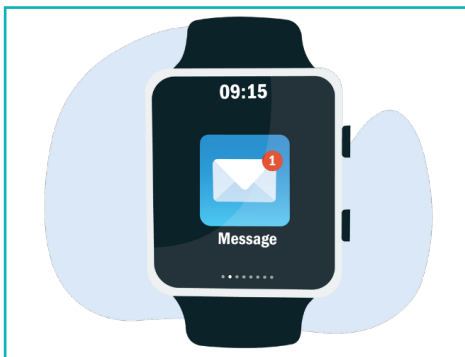


Image credits: Meta

- **Chatbot:** It is a software program that simulates human-like conversations through voice commands.



- **IoT Application Development:** Apps such as Facebook, and IoT devices such as smartwatches, health monitors, etc. can be built using Node.js.



Smart Watch

- **REST API:** Various APIs such as Weather Forecasting API, and Maps API can be built using Node.js.



Applications of Node

- It offers exceptional performance and facilitates the development of scalable web applications. 'Scalability' refers to how easily a web app can be modified/upgraded to accommodate a growing user base and new features.
- The node package manager(npm) which comes with Node, offers a vast selection of packages that can be used to enhance the functionality of Node.
- Node is portable. It is compatible with all major platforms such as Windows, Mac, Solaris, and Linux.

Some Popular Applications Powered by Node.js:



Trademark: Netflix, Inc., LinkedIn Corporation, UBER TECHNOLOGIES, INC., Medium Corporation

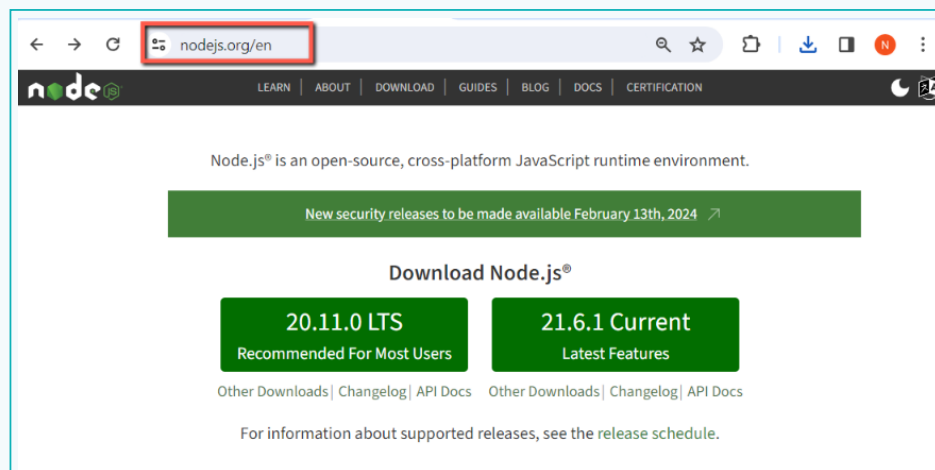
- **Netflix:** It is a video-streaming web application with millions of subscribers. Users can stream movies, television shows, and documentaries from anywhere in the world.
- **LinkedIn:** It is a social media website mainly used by professionals to connect with other working professionals. This can serve the purpose of seeking jobs, hiring by companies, and sharing professional accomplishments.
- **Uber:** It is an organisation that provides access to cars, bikes, and other forms of public transportation, such as autos on hire to get from one place to another.
- **Medium:** It is an immensely popular social publishing platform, with millions of users worldwide. It offers its users the liberty to read and publish stories, articles, and poems on any subject matter.

Activity

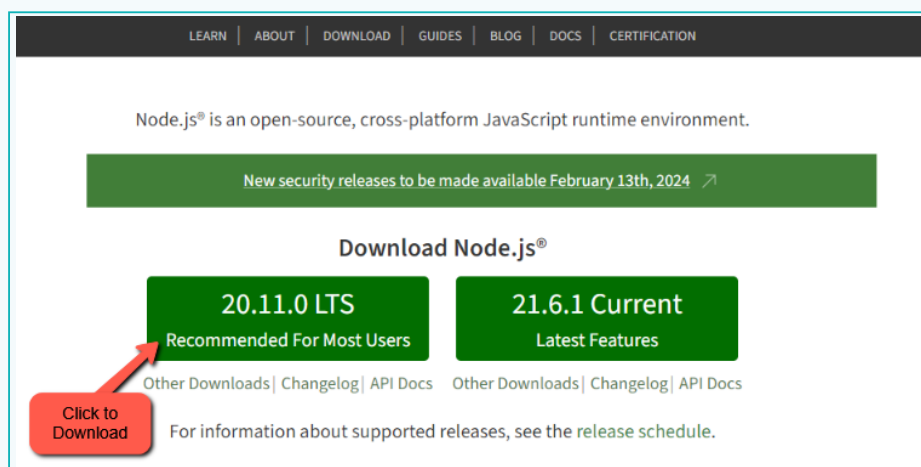
Before we start building a Node Application, it is necessary to set up a local Node Environment in our system. Since Node.js is not available on our systems by default, download the application and install it.

Download and Install Node.js

- 1 Open 'nodejs.org' on your browser. You will be directed to the official Node website.

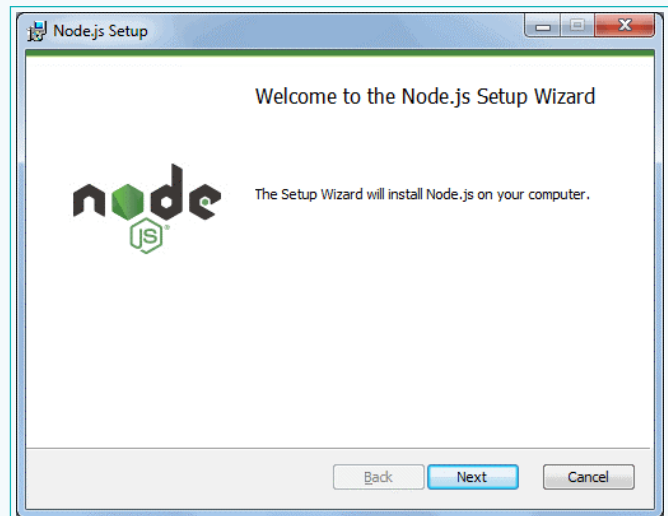


- 2 Click on the button under 'Download Node.js' heading to download Node.js



Activity

- 3 Once the download is complete, install the application.
- 4 Click on 'Next' to read and accept the License Agreement, and then click 'Install'.
- 5 It will install Node.js on your computer quickly. Finally, click 'Finish' to complete the installation.



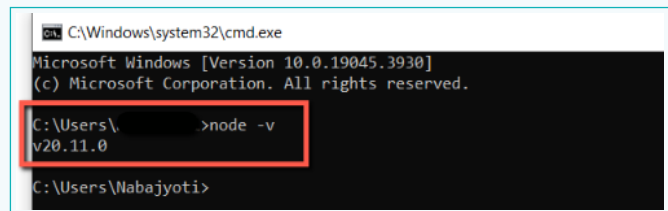
Verify the Installation

- Once you have installed Node.js on your computer, you can verify it by opening the 'command prompt' and typing "node -v".

Note:

To access the 'Command', simply search for it in the 'Start Menu'.

- If Node.js is installed successfully, this command will display the version of Node.js installed on your machine, as shown below:



The next step would be to create your project.

Note:

The terms package, tools, or libraries are interchangeable and refer to the same item.

Exercise

State if the statements are True or False

1 Node.js is a programming language.

▲ _____

2 Node is portable and available for all major operating systems, such as Windows, Mac, and Linux.

▲ _____

Activity

Create a Node Project

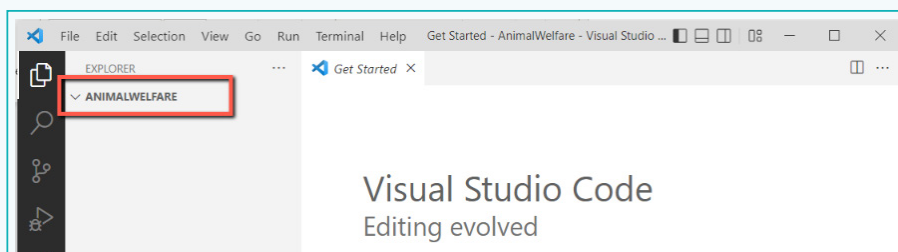
'Node.js' by default, does not include all the packages or modules required to create different types of web applications. This is done to maintain the size of Node.js and ensure it is lightweight.

To get the necessary tools required for creating a Node Application, we use the 'Package Manager'. A '**Package Manager**' is a tool (utility) that we use to download and install packages/modules that are required for application development.

The 'npm' or 'Node Package Manager' is the default package manager that comes with the Node.js installation file and is installed automatically along with it. We can download and install any necessary package for developing a comprehensive node application using npm.

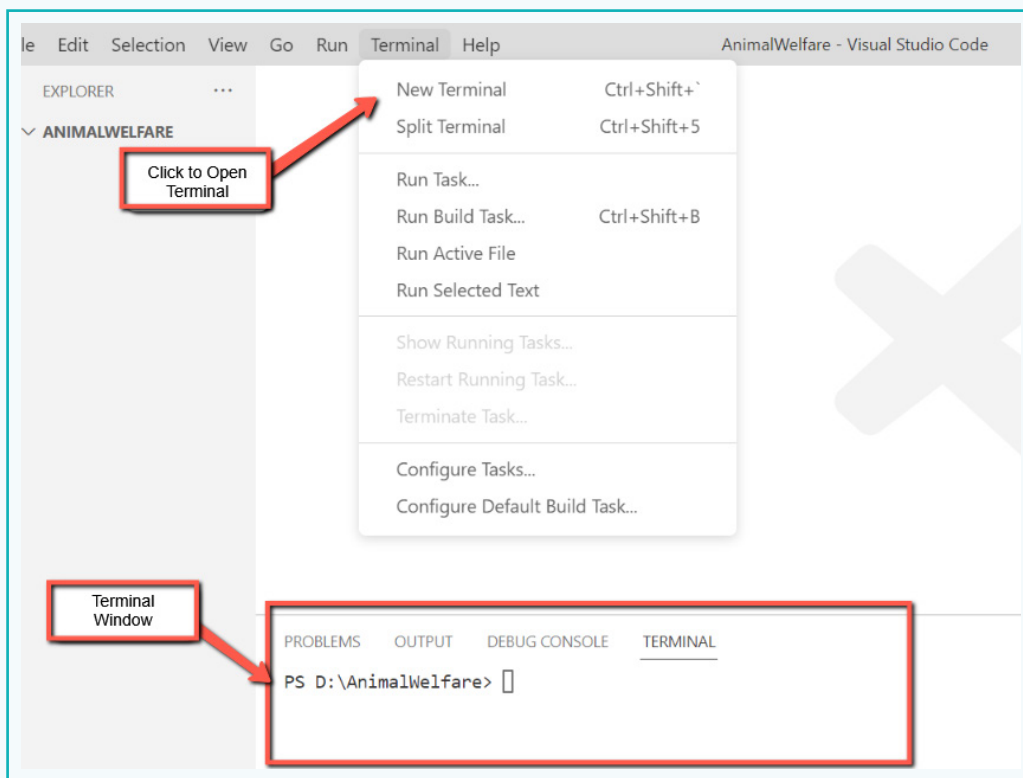
Now that we are familiar with npm, we can create a Node Project.

1 Create a folder "AnimalWelfare" and open it in VS Code.

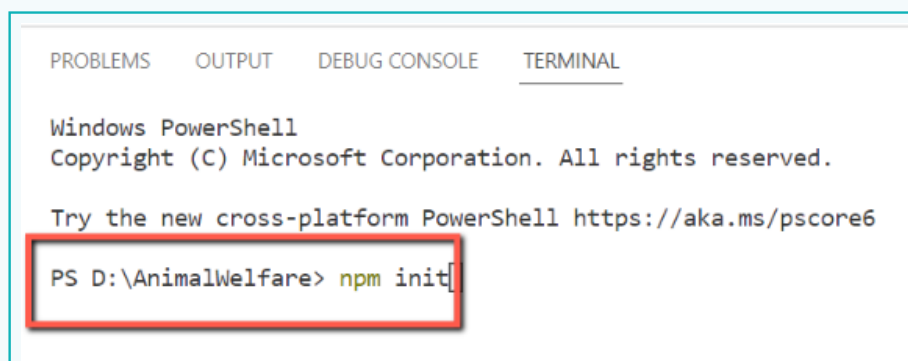


Activity

- 2 Open the Terminal available in VS Code by clicking on the 'Terminal' tab and selecting the 'New Terminal' option. This will open the Terminal window at the bottom of the screen.



- 3 Type the command “**npm init**” in your terminal and then press ‘Enter’. ‘npm init’ is a command used to initialise and create a Node.js project.



Activity

- 4 Press 'Enter', and it will provide you with a variety of options. We will keep the default options and continue pressing the 'Enter' key until the final step.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

Press ^C at any time to quit.
package name: (animalwelfare)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
keywords:
author:
license: (ISC)
About to write to D:\AnimalWelfare\package.json:

{
  "name": "animalwelfare",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC"
}

Is this OK? (yes)
PS D:\AnimalWelfare>
```

- 5 A new file 'package.json' appears inside the folder after the command ends.
- 6 This file is created by the 'npm init' command and contains basic information related to our project.



```
EXPLORER  ...
ANIMAL
package.json

package.json X
package.json > ...
1 {
2   "name": "animalwelfare",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {
7     "test": "echo \"Error: no test specified\" && exit 1"
8   },
9   "keywords": [],
10  "author": "",
11  "license": "ISC"
12 }
13
```

Activity

Note:

There is no need to work or change settings in the package.json file. This file will be used by Node to store any changes or updates we make to our project.

Execute the Node Application

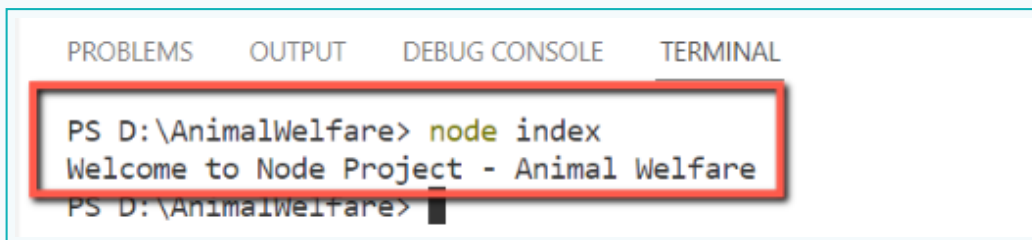
- 1 Create a file 'index.js' in this folder and write a basic 'console.log()' statement in this file.



The screenshot shows the VS Code interface. In the Explorer on the left, a folder named 'ANIMALWELFARE' is expanded, showing 'index.js' and 'package.json'. The 'index.js' file is selected. The Editor on the right shows the content of 'index.js':

```
1 console.log("Welcome to Node Project - Animal Welfare")  
2  
3
```

- 2 To execute our node application, we need to open the terminal and type the command - '**node index**'.
- 3 If the output is displayed as shown, the Node project is working fine.



The screenshot shows the VS Code interface with the Terminal panel open. The terminal shows the command 'node index' being executed in the directory 'D:\AnimalWelfare'. The output is 'Welcome to Node Project - Animal Welfare'.

```
PS D:\AnimalWelfare> node index  
Welcome to Node Project - Animal Welfare  
PS D:\AnimalWelfare>
```

Our Node Project has been successfully executed.

Installing the Express Framework

'Express' is a Node.JS Web Framework that provides various features to develop web applications quickly and using less amount of code.

Note that we can build a Node Application, even without using the Express Framework. Using this framework simply makes the development process easier and more efficient.

However, 'Express' is not available by default. We need to install it using npm.

The command for installing 'Express' is - '**npm install express**'.

Activity

- Open the 'Terminal' and write the command – “npm install express”.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL

PS D:\AnimalWelfare> node index
Welcome to Node Project - Animal Welfare
PS D:\AnimalWelfare>
PS D:\AnimalWelfare>
PS D:\AnimalWelfare> npm install express

added 57 packages, and audited 58 packages in 2s

7 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS D:\AnimalWelfare> █
```

‘Express’ is now installed and ready to use.

Use the Express to Create a Node.JS Web Server

When we type the name of a website in the browser, the browser sends a request to the server. The web server software installed on the server processes the request and sends back the output (response) to the browser. Once the response is received by the browser, it displays the content (web pages) for the user.

Node.js provides the facility to create our own web server by using the ‘Express Framework’, which can handle all incoming requests.

- 1 Include the ‘Express’ module in the application using the ‘require’ function and store it in a variable ‘express’.
- 2 Next, create an ‘express’ object using the ‘express()’ method and store it in a variable ‘app’.

```
JS index.js  X
JS index.js >
1  const express = require('express')
2
3  const app = express()
4
5
```

Activity

Note:

Remove the 'console.log()' statement from the 'index.js' file.

We cannot handle incoming requests from a client by simply creating a Node web server. We also need to configure our web server to 'listen' for incoming connections (requests). The term '**listen**' refers to the functionality where we make our web server wait for any incoming requests.

- 3 We can use the 'listen()' method to make our web server listen. It belongs to the 'express' framework. The 'listen()' method requires a 'port number'.

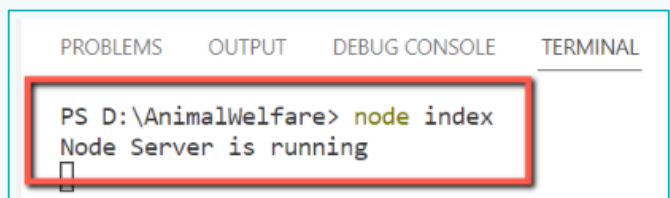
```
JS index.js
JS index.js > ...
1  const express = require('express')
2
3  const app = express()
4
5
6  app.listen(3000, function() {
7    console.log("Node Server is running")
8  })
9
10
```

Note:

The 'app.listen()' function has 2 parameters. One is the 'port number' (3000, in our case), while the second is a 'callback' function. We use the 'callback' function to define what should happen once the connection is established at the server (in our case, a simple console.log() statement).

- 4 Execute your application by typing the command: "**node index**" on the terminal.

- 5 If everything is done correctly, you will get the output- "**Node Server is running**".



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
PS D:\AnimalWelfare> node index
Node Server is running
```

Exercise

■ Fill in the blanks.

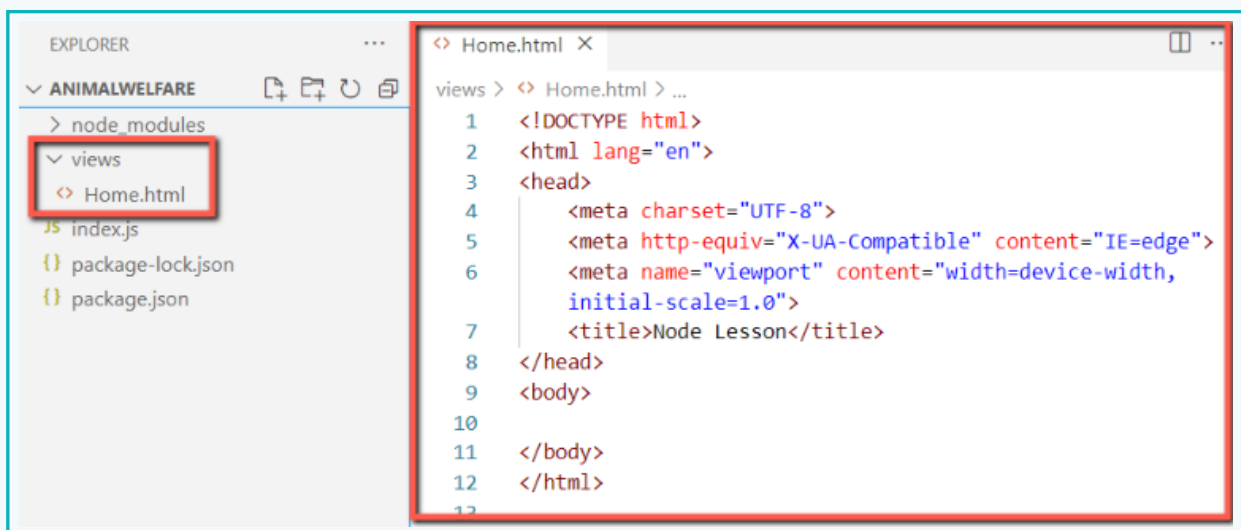
- 3 A popular web framework, used along with Node.js is _____.
- 4 Node.js provides the facility to create our own web server by using the _____ framework.

Activity

■ Create a Web Page in the Application

Now, we will create a webpage using HTML and CSS, and later, we will see how we can view that page in the browser.

- 1 To store the HTML files, we will create a folder and call it 'views'.
- 2 Inside the folder, create a file and name it "Home.html".



Activity

- 3 Create a 'Navigation' bar using the `<nav>` and `<ul>` tags and using the `<a>` tag inside the `<li>` tag to create the Navigation Bar items.

```
<> Home.html X
views > <> Home.html > ...
6   <meta name="viewport" content="width=device-width, initi
7   <title>Node Lesson</title>
8   </head>
9   <body>
10
11   <nav>
12     <ul>
13       <li> <a href=" " > Home </a> </li>
14       <li> <a href=" " > About Us </a> </li>
15       <li> <a href=" " > Register </a> </li>
16       <li> <a href=" " > Our Members </a></li>
17     </ul>
18   </nav>
19
20 </body>
21 </html>
```

Exercise

Tick mark ☒ the correct answer

- 5 Which of the following commands is used to execute a Node project?

- | | | | |
|---------------|--------------------------|---------------|--------------------------|
| a. npm start | ☐ | b. node index | ☐ |
| c. node start | ☐ | d. npm index | ☐ |

Activity

Define Routes

In a normal HTML, CSS website, a webpage can be directly viewed on a browser by double-clicking on the HTML file. However, when working with Node.JS, to view a webpage, we need to associate it with 'Routes'.

'**Routes**' refers to a URL (or path) and a specific HTTP request method (GET, POST). To view a particular page in a Node application, it is necessary to associate that page with a specific URL or route (like '/', 'home', '/about', etc). This ensures that the required webpage is displayed in the browser when a user visits that URL. This technique of associating a webpage to a particular route is known as '**Routing**'.

A Route definition has the following syntax:

<app or router>.<METHOD_NAME>(<PATH>, <HANDLER>)

Here,

- The '**app**' is an instance of 'express'.
- The '**router**' is an instance of an Express router.

Here, we can use either 'app' or 'router'.

- '**METHOD**' is an 'HTTP' request method (**GET, POST**).
- '**PATH**' is the location of the file on the server.
- '**HANDLER**' is the function executed when the route is matched.

Now, let us define a route that will be associated with the 'Home.html' file.

- 1 Open the 'index.js' file and define the route.



```
<> Home.html  JS index.js  X
JS index.js > ...
1  const express = require('express')
2
3  const app = express()
4
5  app.get('/', function(req,res) {
6
7  })
8
9  app.listen(3000, function() {
```


Activity

Code Explanation:

- **app.get():** Indicates that this route uses the 'GET' method.
- **'/ ':** Refers to the 'root' URL. For our application, ' / ' refers to localhost:3000.
- **function(req,res) { ... }:** This declares a callback function that will be executed when a request to this route is made. 'req' and 'res' are default parameters provided by Node.js that are available in this function.

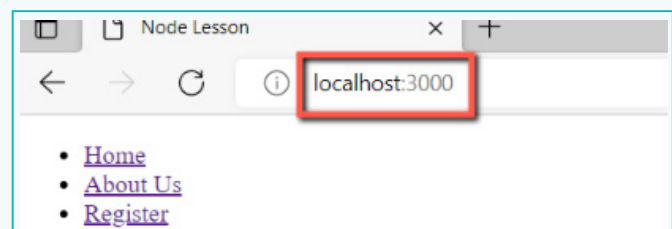
At this point, we have just declared a route. Our main purpose is to display the 'Home.html' page, and that's what we need to define inside this function.

- 2 To display an HTML page, Node.js provides the 'res.sendFile()' function.
- 3 Inside the 'sendFile()' method, specify the location of the 'Home.html' page.

```
index.js
JS index.js > ...
1  const express = require('express')
2  const app = express()
3
4  app.get('/', function(req,res) {
5    res.sendFile('D://AnimalWelfare/views/Home.html')
6  })
7
8  app.listen(3000, function() {
9    console.log("Node Server is running")
10 })
11
```

- 4 Run your application using the command- "node index" on the terminal.
- 5 Open the browser and type "localhost: 3000" on it.

- 6 The 'Home.html' page should be visible now.



Exercise

■ Tick mark ☒ the correct answer

6 Which of the following represents the 'root' route?

- | | | | |
|-----------|--------------------------|------------|--------------------------|
| a. "/" | ☐ | b. "/root" | ☐ |
| c. "root" | ☐ | d. "/home" | ☐ |

Activity

Use CSS to Design the Home Page

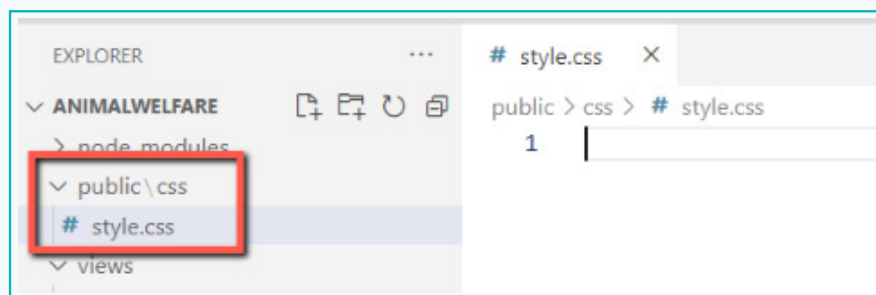
To design our page, we will require CSS, but we cannot just link an external CSS file using the `<link>` tag in Node.js.

Files such as CSS files, images, scripts, etc. are usually referred to as '**Static Files**', as their contents remain static(fixed). Such files are not generated by the server but rather must be sent by the server to the client when requested. This technique is referred to as "**Serving Static Files**".

Node.js does not serve static files by default, and we need to configure an application to do so.

The steps involved in making an external CSS file available are:

- 1 First, we will create a folder named "public".
- 2 Create another folder inside the 'public' folder and name it "css".
- 3 Inside the 'css' folder, create the 'style.css' file.



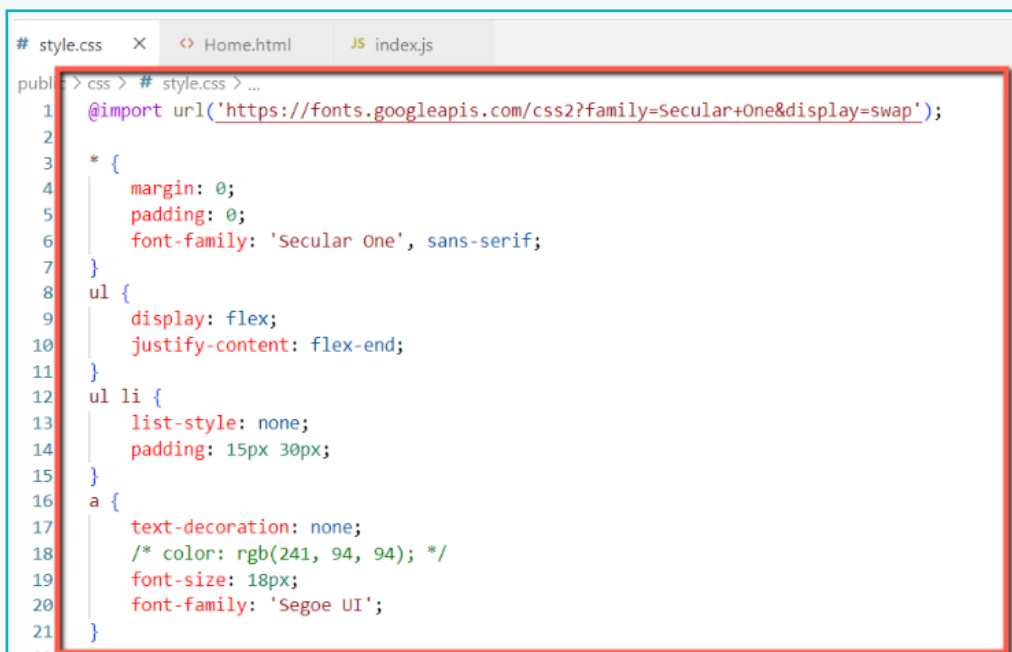
Activity

- 4 Next, in the 'index.js' file, use the 'express.static()' function to include the location of our static files in our application.
- 5 Our static files are located inside the 'public' folder. Specify this folder.



```
# style.css JS index.js X
JS index.js > ...
1  const express = require('express')
2  const app = express()
3
4  app.use(express.static('public'));
5
6  app.get('/', function(req,res) {
7    res.sendFile('D://AnimalWelfare/views/Home.html')
8  })
```

- 6 Open the 'style.css' file and add the following CSS Code.



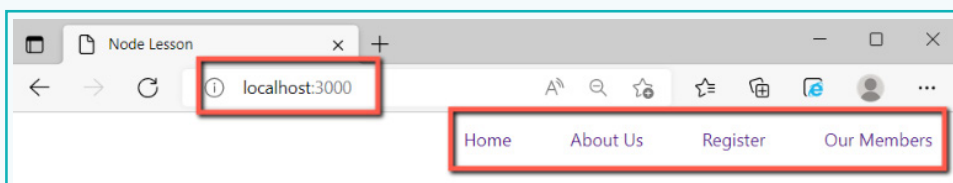
```
# style.css X <> Home.html JS index.js
publ > css > # style.css > ...
1  @import url('https://fonts.googleapis.com/css2?family=Secular+One&display=swap');
2
3  * {
4    margin: 0;
5    padding: 0;
6    font-family: 'Secular One', sans-serif;
7  }
8  ul {
9    display: flex;
10   justify-content: flex-end;
11 }
12 ul li {
13   list-style: none;
14   padding: 15px 30px;
15 }
16 a {
17   text-decoration: none;
18   /* color: rgb(241, 94, 94); */
19   font-size: 18px;
20   font-family: 'Segoe UI';
21 }
```

Activity

- 7 In the final step, we must include this 'style.css' file in our 'Home.html' page.

```
> Home.html X
views > <> Home.html > html > body > nav
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta http-equiv="X-UA-Compatible" content="IE=edge">
6      <meta name="viewport" content="width=device-width, initial-
7      <title>Node Lesson</title>
8
9      <link rel="stylesheet" href="/css/style.css">
10
11 </head>
12 <body>
```

- 8 Now, if you visit the browser, you will see the CSS properties applied to the page.



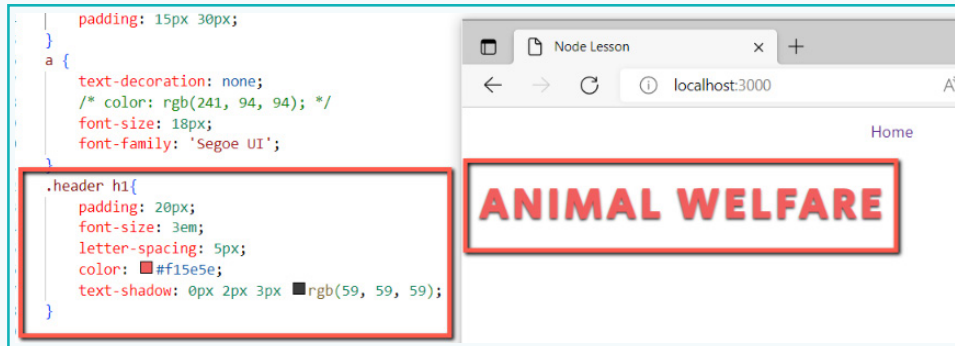
Add Remaining Content to the Home Page

- 1 Create a section and give it a class 'header'.
- 2 Inside, create a heading using the <h1> tag.

```
20      </ul>
21    </nav>
22
23    <section class="header">
24      <h1> ANIMAL WELFARE </h1>
25    </section>
26
27  </body>
```

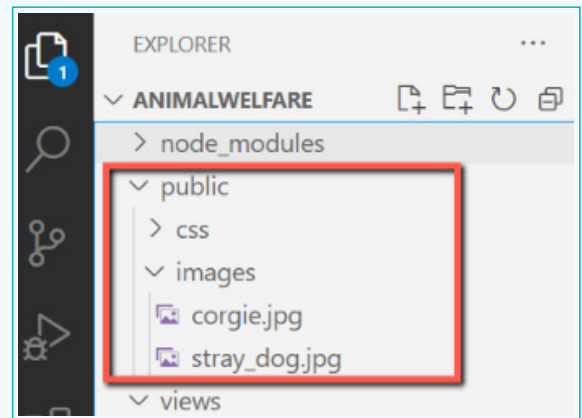
Activity

- 3 Add some CSS to design it.

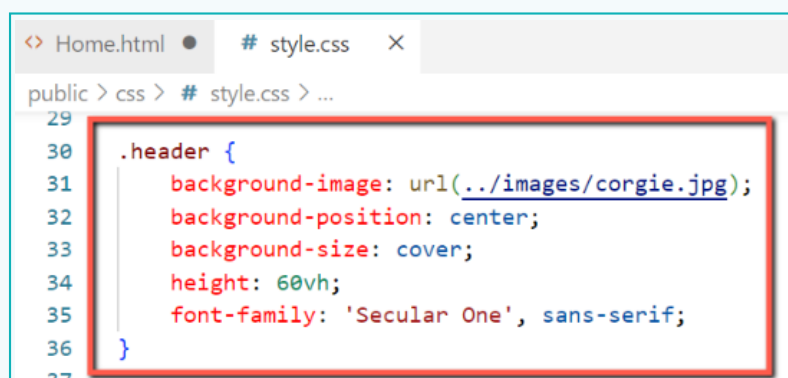


- 4 Add an image as a background for the 'header' class section.

- 5 Create a folder named "images" inside the 'public' directory and store the image you want to use there.

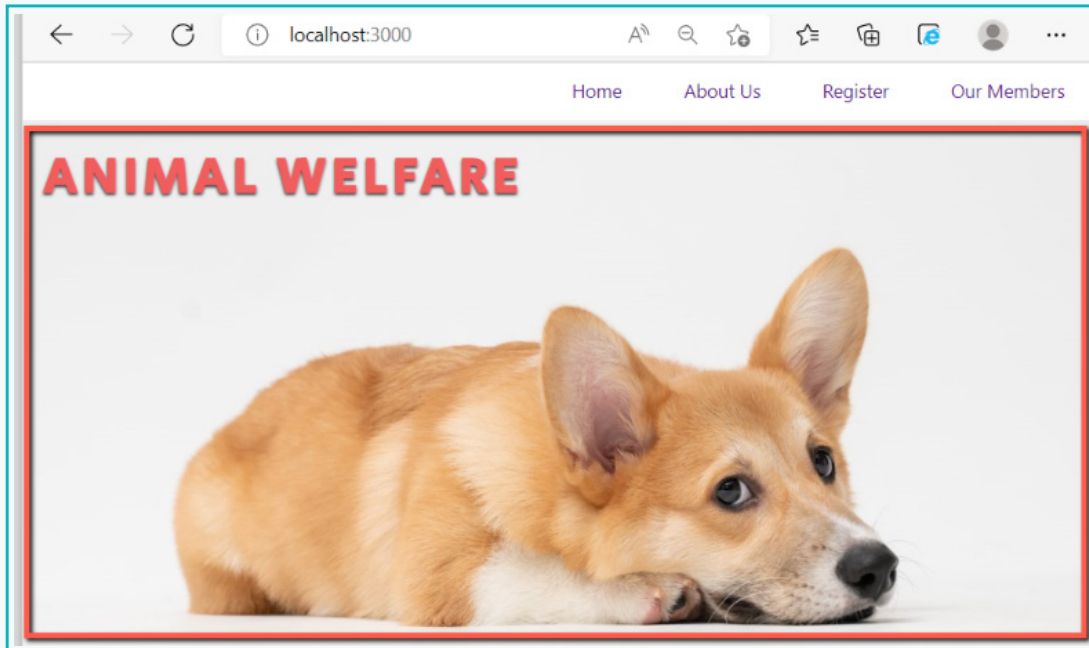


- 6 Set some more CSS properties for the 'header' class section.



Activity

- 7 Check the output on the browser.



- 8 Next, create the 'About Us' section, as shown in the figure below.

```
<section class="aboutus_section">
  
  <div class="aboutus_content">
    <h2> ABOUT US </h2>
    <p>
      Our team consists of like-minded individuals who share a common vision
      for improving the lives of abandoned, abused, and neglected animals.
      With the help of our members and volunteers, we provide food, shelter,
      and care for these animals. <br>
      Also, we conduct workshops to raise awareness about Animal Protection
      Laws and much more.
    </p>
  </div>
</section>
```

Activity

- 9 Add a few CSS properties for the 'About Us' section and its contents.

```
<> Home.html ● # style.css ●
public > css > # style.css > ...
37
38
39 .aboutus_section {
40     display: flex;
41     width: 80%;
42     margin: auto;
43     padding: 30px;
44 }
45 .aboutus_section img {
46     width: 35%;
47     border-radius: 5px;
48 }
49 .aboutus_content {
50     padding-left: 30px;
51 }
52 .aboutus_content h2 {
53     font-size: 2.2rem;
54     color: #f15e5e;
55     margin-bottom: 20px;
56     border-bottom: 1px solid grey;
57 }
58 .aboutus_content p {
59     line-height: 40px;
60     font-family: 'Segoe UI';
61     font-size: 18px;
62 }
63
```

Activity

- 10 View the output on the browser.



ABOUT US

Our team consists of like-minded individuals who share a common vision for improving the lives of abandoned, abused, and neglected animals. With the help of our members and volunteers, we provide food, shelter, and care for these animals.

Also, we conduct workshops to raise awareness about Animal Protection Laws and much more.

Our Home page is now complete.

Output

ANIMAL WELFARE



ABOUT US

Our team consists of like-minded individuals who share a common vision for improving the lives of abandoned, abused, and neglected animals. With the help of our members and volunteers, we provide food, shelter, and care for these animals.

Also, we conduct workshops to raise awareness about Animal Protection Laws and much more.

We have learnt that Node.js can be used to create dynamic and attractive server-side websites or web applications.

Exercise

State if the statements are True or False

7 The package installer '**npm**' comes pre-installed with Node.js.

 _____

8 '**console.log()**' method is used to display output on the webpage.

 _____

Fill in the blanks.

9 A popular package manager '**npm**' stands for _____.

10 To create a Node Project, we need to execute the _____ command.

Tick mark ☒ the correct answer

11 Which of the following functions is used to specify the location of the static files in a Node Application?

a. App.static() ☐

b. Express.static() ☐

c. Router.static() ☐

d. App.express.static() ☐

The emergence of Node.js has made the web app development process very easy. It is highly accessible. Many popular applications are created using Node. It offers limitless opportunities to explore the expansive world of web apps.



Tech Flash

- **Node.js** : An Open-source runtime environment used for building server-side applications using JavaScript.
- **Express** : A small and flexible Node.js web framework that provides various features for developing web applications.
- **Web Server** : It refers to computer software that is responsible for handling incoming requests from clients(users), processing them and sending back the generated response(output).